LLM = Large Language Model

A Large Language Model **is a type of** artificial intelligence (AI) **model designed to** understand, generate, and process human language. It is a type of **machine learning model**, specifically based on **deep learning** using **neural networks**.

Working principle:

- The model is trained on **massive text datasets** (books, articles, websites, etc.).
- It learns **patterns in language**, such as grammar, meaning, and context.
- Once trained, it can **generate human-like text**, **answer questions**, **summarize**, **translate**, **analyze sentiment**, and more.

Examples of LLMs:

- **GPT-5** (by OpenAI)
- **Gemini** (by Google DeepMind)
- **Claude** (by Anthropic)
- **LLaMA** (by Meta)

Mathematical overview:

An LLM is a parameterized probability model that estimates

$$P(w_t / w_1, w_2, \ldots, w_{t-1})$$

This means it predicts the next word in a sequence given all previous words.

Objective of an LLM
An **LLM** is a probabilistic model that tries to estimate the likelihood of a sequence of words:
$P(w_1, w_2, \ldots, w_T)$

where w_i is the ith word (or token) in the sequence.
By the chain rule of probability:

$$P(w_1, w_2, \ldots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, w_2, \ldots, w_{t-1})$$

So the model's goal is to predict the next token given all previous tokens.

CamScanner

Problem:

Show the full mathematical flow of a mini LLM with 1 layer, 3 tokens, and tiny dimensions (2D) for the following sentence fragment to predict the next word:

"The cat sat"

Solution:

Given that,

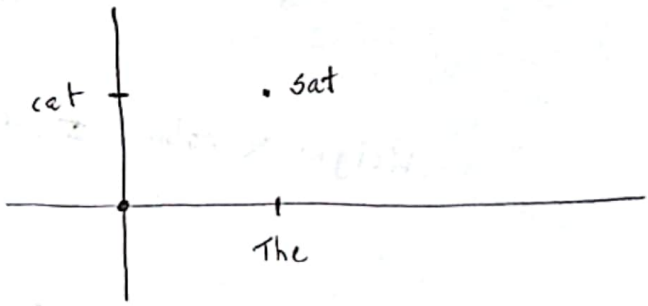
- Number of tokens (T) = 3
- Number of layers (L) = 1

Let us consider

- Embedding dimension (d) = 2
- Attention dimension (d_k) = 2

Step 1: Token Embedding

Token	Embedding x_t
The	[1, 0]
cat	[0, 1]
sat	[1, 1]



The input matrix is:

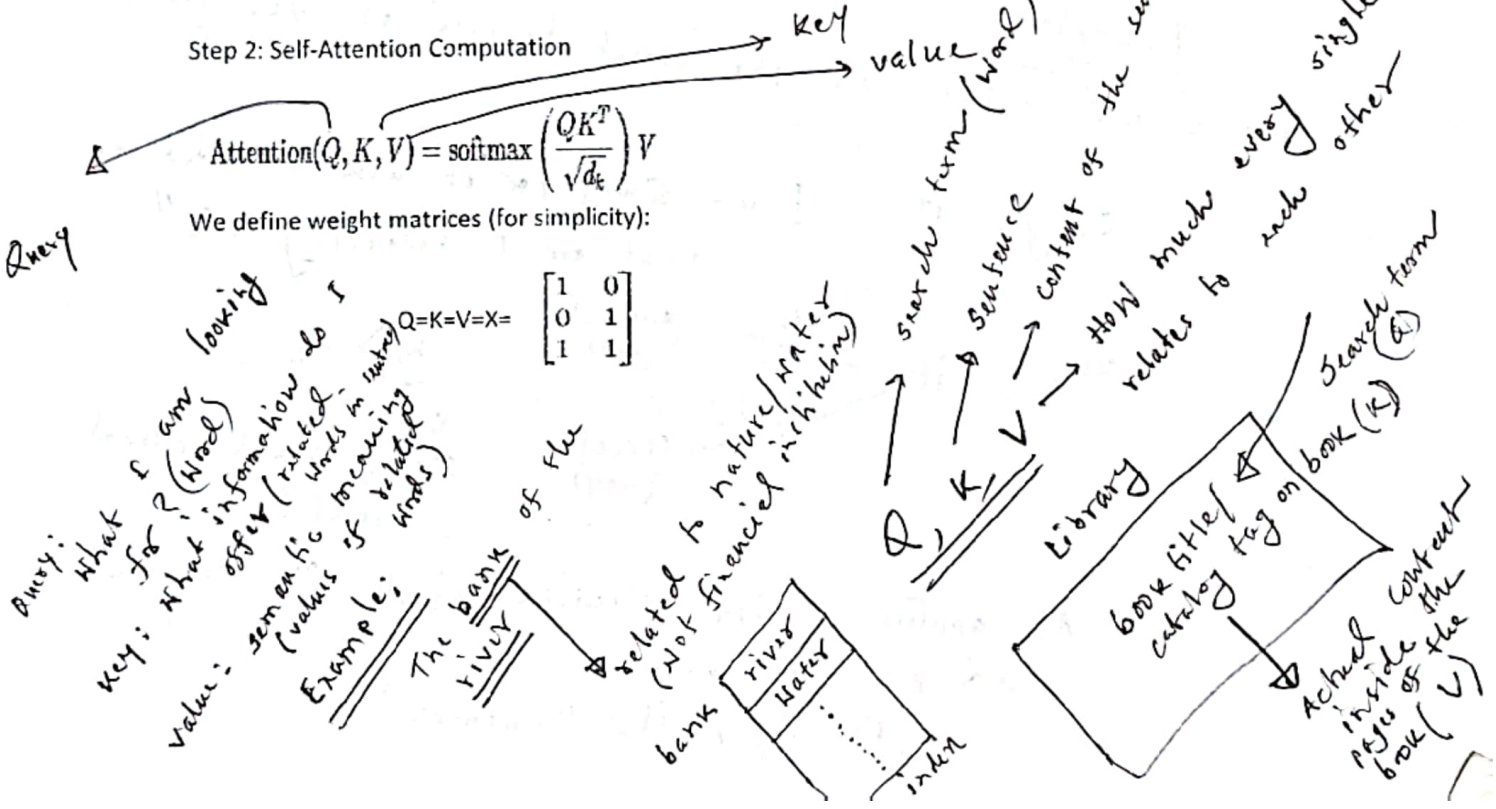
$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

Step 2: Self-Attention Computation

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

We define weight matrices (for simplicity):

$$Q=K=V=X= \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$



$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \quad K^T = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

(a) Compute QK^T

3×2 · 2×3 → 3×3

$$QK^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}$$

Then divide by $\sqrt{d_k} = \sqrt{2} \approx 1.414$: (Attention dimension)

$$\frac{QK^T}{\sqrt{2}} = \begin{bmatrix} 0.71 & 0 & 0.71 \\ 0 & 0.71 & 0.71 \\ 0.71 & 0.71 & 1.41 \end{bmatrix}$$

(b) Apply Softmax row-wise

For each row, we apply

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

For row 1: [0.71, 0, 0.71]

Exponentiation → [2.03, 1, 2.03]

Normalization → [0.41, 0.18, 0.41]

Similarly, applying softmax, we obtain the following:

Token	Softmax weights (attention to [The, cat, sat])
The	[0.41, 0.18, 0.41]
cat	[0.18, 0.41, 0.41]
sat	[0.28, 0.28, 0.44]

Similarly

Similarly

(c) Compute Attention Output

Attention (Q, K, V) =weights × V

Since V=X, we multiply each attention row by X:

1. For "The":

$$[0.41, 0.18, 0.41] \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} = [0.82, 0.59]$$

(The)

2. For "cat":

$$[0.18, 0.41, 0.41]X = [0.59, 0.82]$$

(cat)

3. For "sat":

$$[0.28, 0.28, 0.44]X = [0.72, 0.72]$$

(sat)

CamScanner

Thus attention outputs:

$$A = \begin{bmatrix} 0.82 & 0.59 \\ 0.59 & 0.82 \\ 0.72 & 0.72 \end{bmatrix}$$

Step 3: Feed-Forward Layer

Each token vector passes through a simple fully connected feed-forward layer:

$$h_t = \mathrm{ReLU}(A_t W_1 + b_1)$$

Let

$$W_1 = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}, \quad b_1 = [0, 0]$$

For "The":

$$h_1 = [0.82, 0.59] \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$$

= [1.41, -0.23]=> ReLU(=[1.41, -0.23])= =[1.41, 0]

For "cat":

$$h_2 = [0.59, 0.82] \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$$

= [1.41, 0.23]=> ReLU(=[1.41, 0.23])= =[1.41, 0.23]

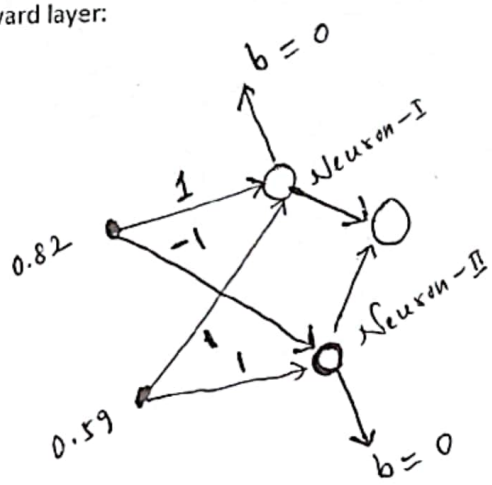
For "sat":

$$h_3 = [0.72, 0.72] \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$$

= [1.44, 0]=> ReLU(=[1.44, 0])= =[1.44, 0]

$$H = \begin{bmatrix} 1.41 & 0 \\ 1.41 & 0.23 \\ 1.44 & 0 \end{bmatrix} \begin{matrix} \leftarrow h_1 \\ \leftarrow h_2 \\ \leftarrow h_3 \end{matrix}$$

sat



Neuron - I

$(0.82 * 1 + 0.59 * 1) + b$

$= 1.41 + b = 1.41$

$(0.82 * (-1) + 0.59 * 1)$

Neuron-II

$+ b$

$= -0.23 + 0$

$= -0.23$

Relu = $\begin{cases} \max(0, x) \\ 0 \end{cases}$

1.41

x 1 0

x 1 0

-0.23

Step 4: Output Probability (Softmax over Vocabulary)

Assume a tiny vocabulary: {on, dog, under}.

Consider the output weight matrix

$$W_O=\begin{bmatrix}1 & 0\\ 0 & 1\\ 1 & 1\end{bmatrix}$$ → Assumed

For the last token "sat"

$[1.44, \quad 0]$ ← h_1

$$\text{logits}=\left[W_O \times h_t^T\right]^T$$

$$W_O h_t^T=\begin{bmatrix}1 & 0\\ 0 & 1\\ 1 & 1\end{bmatrix} \times [1.44, \quad 0]^T= \begin{bmatrix}1 & 0\\ 0 & 1\\ 1 & 1\end{bmatrix} \times \begin{bmatrix}1.44\\ 0\end{bmatrix}= \begin{bmatrix}1.44\\ 0\\ 1.44\end{bmatrix}$$

Logits = [1.44, 0, 1.44]

Apply softmax:

softmax ([1.44, 0, 1.44])

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

$z_i=1.44$; $e^{zi}=4.22$; $z_i=0$; $e^{zi}=1$; $z_i = 1.44$; $e^{z_i} = 4.22$

Summation= 4.22+1+4.22=9.44

softmax([1.44, 0, 1.44])= [0.45, 0.11, 0.45]

$$\frac{4.22}{4.22+1+4.22} = 0.45 \qquad \frac{1}{4.22+1+4.22} = 0.11 \qquad \frac{4.22}{4.22+1+4.22} = 0.45$$

Interpretation:
The model predicts the next word with:

- P("on") = 0.45
- P("dog") = 0.11
- P("under") = 0.45

highest probability

So it's most likely to choose "on" or "under" as the next token.

CamScanner

ReLU:

The ReLU function can be described mathematically as follows:

$$f(x) = \max(0, x)$$

Where:

- x is the input to the neuron.
- The function returns x if x is greater than 0.
- If x is less than or equal to 0, the function returns 0.

The formula can also be written as:

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

This simplicity is what makes ReLU so effective in training deep neural networks, as it helps to maintain non-linearity without complicated transformations, allowing models to learn more efficiently.

CamScanner